

SISTEMA DE GESTIÓN Y PLANIFICACIÓN OPERACIONAL DE FLOTA Y TRIPULACIÓN

Especificación funcional + ingeniería + arquitectura + prototipo base

*Documento de levantamiento y diseño — Versión 0.1
01 septiembre 2026*

1. Propósito del documento

Este documento consolida el levantamiento realizado sobre la operación de una empresa de buses y define una base común para que un desarrollador, equipo técnico o una IA pueda diseñar y construir la aplicación sin perder las reglas operacionales conocidas. El documento separa explícitamente lo confirmado de lo que todavía debe validarse.

OBJETIVO DEL MVP

La primera versión no debe intentar digitalizar toda la empresa. El núcleo debe resolver planificación de posturas, asignación de máquinas, asignación de tripulación, conflictos, mantenimiento, fallas y corridas. WhatsApp, GPS y sistemas de ventas se diseñan como integraciones posteriores.

2. Problema que se busca resolver

- Operaciones actualmente coordina posturas, buses y tripulación con fuerte dependencia de Excel y comunicaciones manuales.
- La asignación de tripulación es uno de los principales dolores: disponibilidad, relieves, jornadas, descansos, contratos y cambios de última hora.
- Un bus puede fallar y la información operacional requiere coordinación rápida; durante ruta, cuando la falla impide continuar, el conductor contacta por WhatsApp al mecánico más cercano.
- El checklist se realiza únicamente cuando el bus llega a Santiago. Hoy pasa por digitación/revisión y luego llega a Mantenimiento.
- Si un bus no está disponible para una postura, se produce una corrida: otra máquina cubre la postura y las siguientes pueden desplazarse hasta recuperar la secuencia.
- Existe información de GPS en tiempo real y sistemas de ventas, pero los proveedores/API aún deben identificarse antes de integrar.

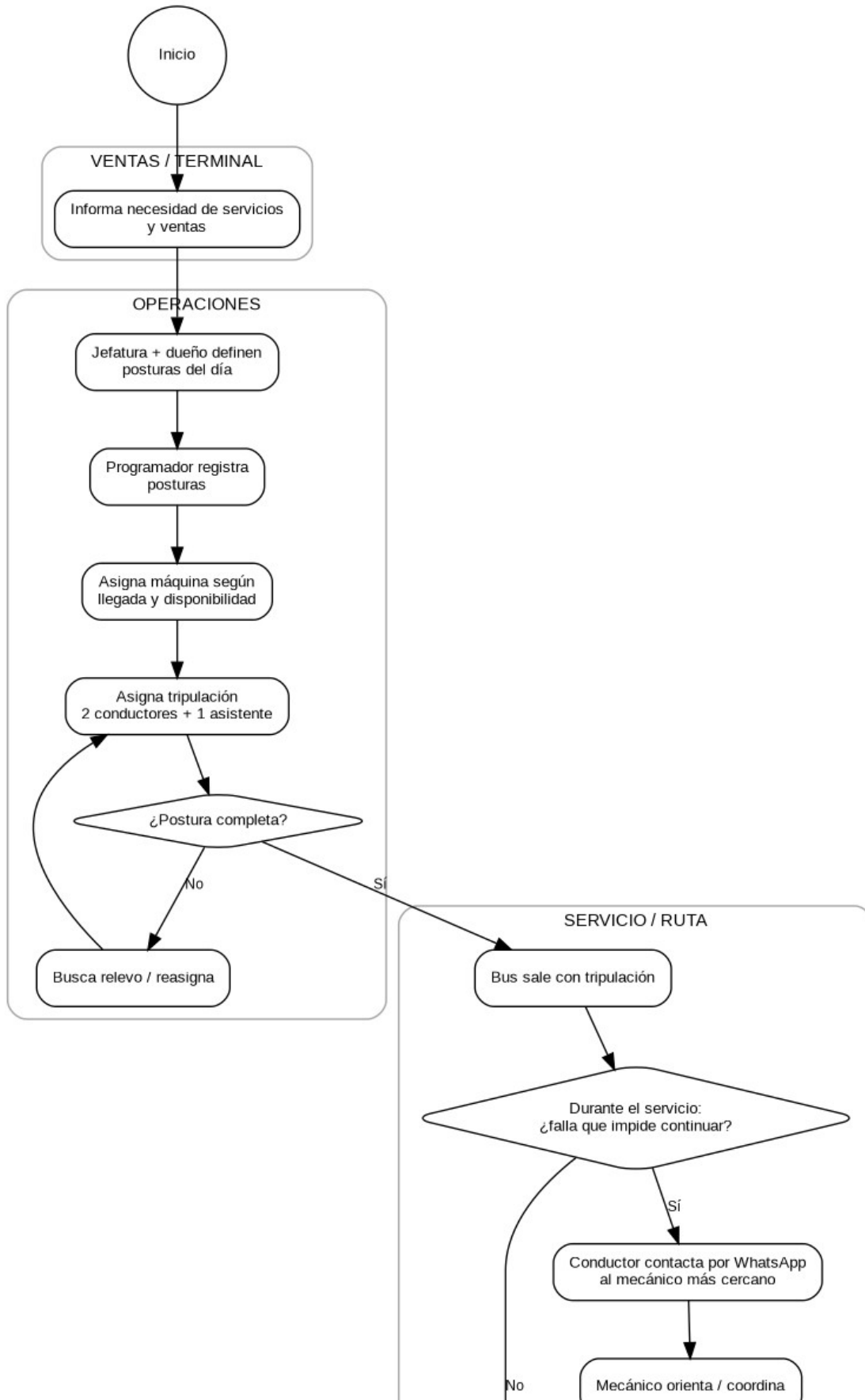
3. Principios de diseño

Principio	Aplicación
Primero proceso, después software	Validar AS-IS con Operaciones antes de cerrar el TO-BE.
No sobrescribir historia	Cambios de bus, tripulación y corridas deben conservar historial.
Reglas configurables	Jornada, descanso y límites contractuales deben parametrizarse y validarse antes de automatizar.
Operación simple	La complejidad queda en el motor; el usuario ve estados, alertas y razones.
Mobile-first para conductores	El conductor debe poder operar desde celular; Operaciones puede usar PC.
Integraciones desacopladas	WhatsApp, GPS y ventas deben entrar por APIs/adaptadores, no contaminar el núcleo.
Auditoría	Toda modificación relevante debe guardar quién, cuándo, qué cambió y motivo.

4. Proceso actual (AS-IS)

Este es el flujo que debe ser validado con el usuario de negocio antes de construir el prototipo definitivo.

PROCESO ACTUAL (AS-IS) — OPERACIÓN DE BUSES



4.1. Reglas operacionales confirmadas

Tema	Regla / situación conocida
Posturas	Se crean diariamente según necesidad/fecha. Jefatura de Operaciones + dueño definen; la necesidad se alimenta desde Ventas/Terminal. Puede haber aumentos si las ventas lo justifican.
Servicio vs postura	Servicio es el tipo de servicio (ej. Semi Cama, Salón Cama, Cama Premium). Postura es el servicio concreto programado para fecha/hora/origen/destino.
Postura sin bus	Sí. Primero puede existir la postura y luego se asigna la máquina según disponibilidad/llegada.
Postura sin tripulación completa	Sí. Normalmente existe un conductor titular/fijo asociado a la máquina y Operaciones completa con relieves.
Tripulación	Normalmente 3 personas: 2 conductores + 1 asistente.
Prioridad	Lunes y martes tienen prioridad los servicios mineros; luego se prioriza la postura más cercana en tiempo.
Cambios de postura	Pueden ocurrir durante el día. Operaciones y Mantenimiento pueden intervenir según el caso.
Comunicación a tripulación	Principalmente WhatsApp; llamadas en urgencias dentro de Santiago.
Checklist	Se realiza una vez por viaje, al llegar a Santiago. El conductor detalla la falla; Mantenimiento decide qué hacer.
Falla en ruta	Si impide continuar, conductor contacta por WhatsApp al mecánico más cercano. El mecánico orienta/coordina. Si no puede continuar, se coordina con Operaciones.
Mantenimiento	Jefe de Mecánicos revisa checklist, considera múltiples fallas, especialidad y disponibilidad, y asigna trabajo.
Pozo	Es el lugar físico de trabajo mecánico donde se estacionan buses para inspección/reparación.
Liberación	Jefe de Mecánicos libera el bus tras revisar el trabajo; en algunos casos se realiza prueba de ruta.
Corrida	Si el bus no puede cumplir una postura, Operaciones lo retira y otra máquina cubre la postura. Las siguientes pueden desplazarse hasta recuperar la secuencia.
Número de servicio	Cada bus tiene un número de servicio que se usa en la hoja de ruta; la relación exacta entre número de servicio, máquina y postura debe validarse.
GPS	Existe SINACH + monitoreo interno con ubicación en tiempo real; proveedor/API pendiente de identificar.
Pasajeros	Los listados son impresos por la agencia/terminal; la información vive en el sistema de ventas de cada oficina. Se identificó el andén como dato útil.

5. Flujos críticos que el sistema debe soportar

5.1. Asignación de una postura

1. Crear postura con fecha, hora, origen, destino y servicio.
2. Permitir que exista sin bus y/o sin tripulación completa.
3. Seleccionar una máquina disponible o futura según llegada prevista.
4. Asignar tripulación.
5. Validar conflictos de conductor/relieve y estado de máquina.
6. Guardar asignaciones con historial, no reemplazarlas silenciosamente.
7. Marcar la postura como lista, con advertencia o bloqueada.

5.2. Conflicto de conductor

La interfaz propuesta debe ocultar la complejidad:

Estado	Comportamiento
● Puede asignarse	No existe conflicto conocido con las reglas activas.
● Advertencia	Existe riesgo o excepción que un usuario autorizado puede revisar.
● Bloqueado	La asignación incumple una regla dura configurada.

- Ejemplos: conductor ya asignado a otra postura; solapamiento horario; descanso insuficiente; límite de jornada/días; otra restricción contractual.

REGLAS LEGALES / CONTRACTUALES

Las reglas laborales/contractuales mencionadas en el levantamiento no deben convertirse en números fijos hasta ser validadas con la empresa y, cuando corresponda, con asesoría laboral. El software debe permitir parametrizarlas.

5.3. Falla durante el servicio

8. Conductor detecta falla.
9. Si puede continuar, continúa el servicio.
10. Si no puede continuar, contacta por WhatsApp al mecánico más cercano.
11. El mecánico orienta/coordina las acciones.
12. Si no puede continuar, se coordina con Operaciones.
13. Operaciones gestiona la continuidad del servicio y, si corresponde, reemplazo/corrida.
14. El evento debe quedar registrado en el sistema aunque la conversación se produzca por WhatsApp.

5.4. Checklist al llegar a Santiago

15. Bus llega a Santiago.
16. Conductor completa checklist.
17. Actualmente el digitador recibe, revisa y digita; luego imprime/entrega a Mantenimiento.
18. Mantenimiento revisa fallas y decide acciones.
19. El nuevo sistema debe eliminar el paso manual de digitación como objetivo del MVP: el conductor registra digitalmente y Mantenimiento recibe el resultado.

DISTINCIÓN CLAVE

El checklist NO se ejecuta durante la ruta. Durante la ruta se debe modelar un INCIDENTE/AVISO DE FALLA separado del checklist.

5.5. Corrida

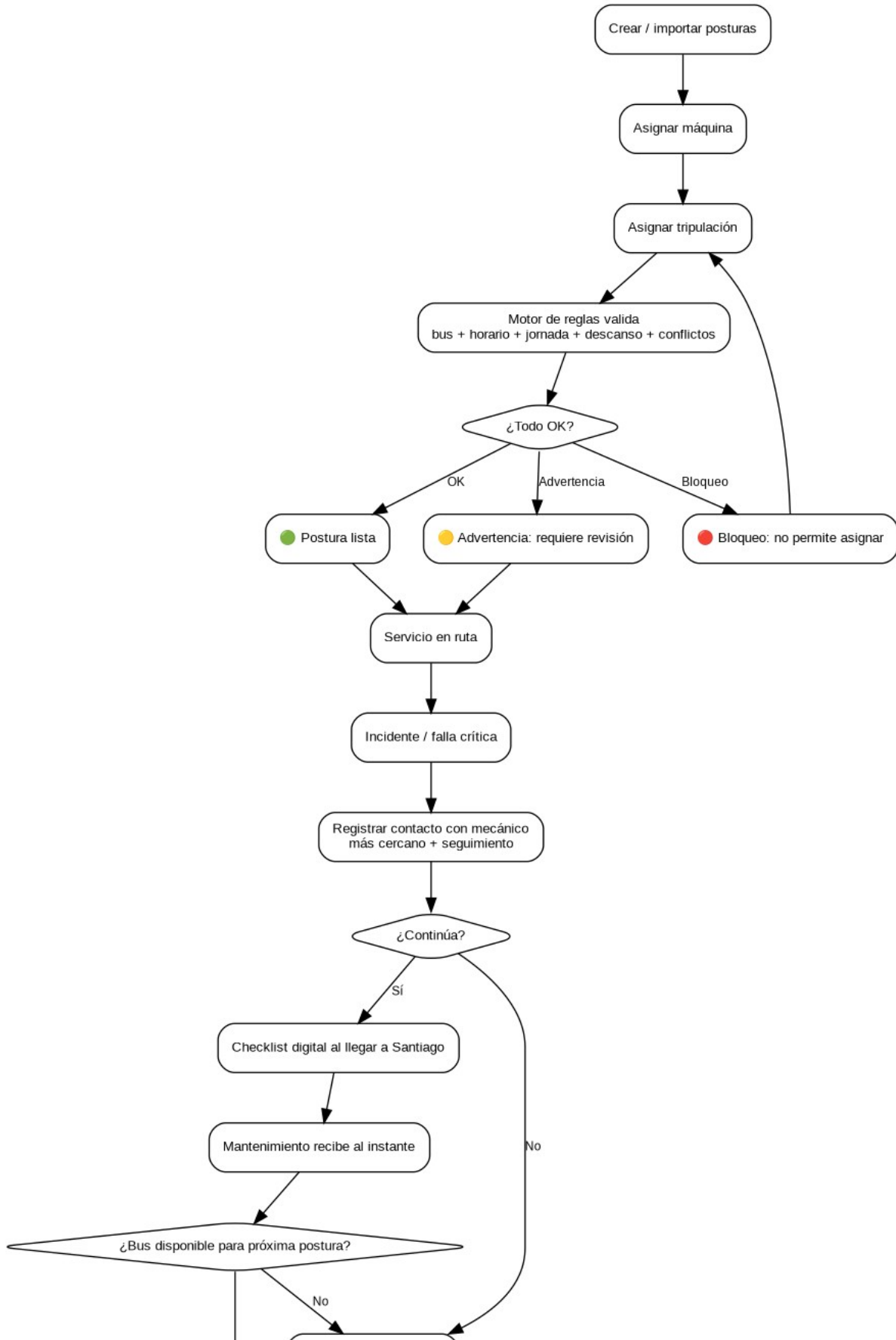
20. Mantenimiento/Operaciones determinan que la máquina no puede cumplir una postura.
21. La postura queda sin su máquina original.
22. Operaciones selecciona una máquina sustituta.
23. El sistema registra el cambio y el motivo.
24. Las posturas posteriores pueden desplazarse/cubrirse en secuencia.
25. Cuando el bus reparado queda liberado, la corrida puede detenerse o reconfigurarse; la lógica exacta de reincorporación debe validarse.

CORRIDA

No se debe sobrescribir el bus original de la postura. Debe quedar una trazabilidad: bus original → bus sustituto → motivo → fecha/hora → usuario → efecto sobre posturas siguientes.

6. Proceso futuro propuesto (TO-BE)

PROCESO PROPUESTO (TO-BE) — SISTEMA DE GESTIÓN

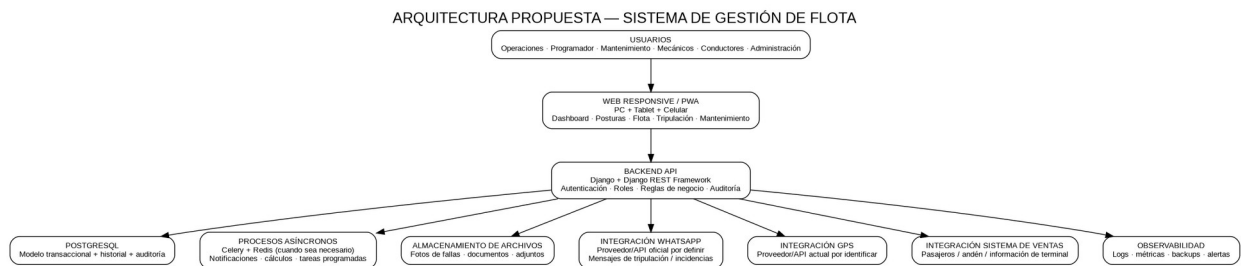


El TO-BE propone que el sistema se convierta en el centro de coordinación, mientras WhatsApp/GPS/ventas funcionan como canales o fuentes integradas. La aplicación no debe obligar a Operaciones a entender el motor de reglas: debe mostrarle el resultado y el motivo.

7. Alcance funcional por módulos

ID	Módulo	Responsabilidad
M1	Posturas	CRUD de posturas; calendario; estados; servicio; origen/destino; prioridad; aumentos; cambios controlados.
M2	Flota	Buses, número interno, patente, número de servicio, estado, ubicación/pozo, disponibilidad.
M3	Tripulación	Conductores, asistentes, titulares, relieves, asignaciones, disponibilidad y conflictos.
M4	Planificación	Vista diaria, semanal, tablero de cobertura, filtros y búsqueda de candidatos.
M5	Motor de reglas	Conflictos de horario, disponibilidad, descanso/jornada parametrizable, estados de bus y restricciones.
M6	Corridas	Crear, visualizar, ejecutar y auditar cambios de máquina y desplazamientos de posturas.
M7	Viajes / servicio	Estado de salida, en ruta, llegada y cierre.
M8	Incidentes	Fallas en ruta, gravedad, continuidad, contacto con mecánico, seguimiento y cierre.
M9	Checklist	Checklist configurable por bus/servicio; captura móvil al llegar a Santiago; fotos; firma si se requiere.
M10	Mantenimiento	Órdenes, fallas, mecánicos, especialidades, pozo, reparación, pruebas y liberación.
M11	Comunicaciones	Plantillas y registro de mensajes; WhatsApp como integración desacoplada.
M12	Pasajeros / terminal	Integración futura con sistemas de venta; pasajeros, andén y estado de embarque.
M13	GPS	Integración futura con proveedor actual; ubicación y trazabilidad.
M14	Administración	Usuarios, roles, catálogos, reglas, auditoría y configuración.

8. Arquitectura de ingeniería



8.1. Stack recomendado

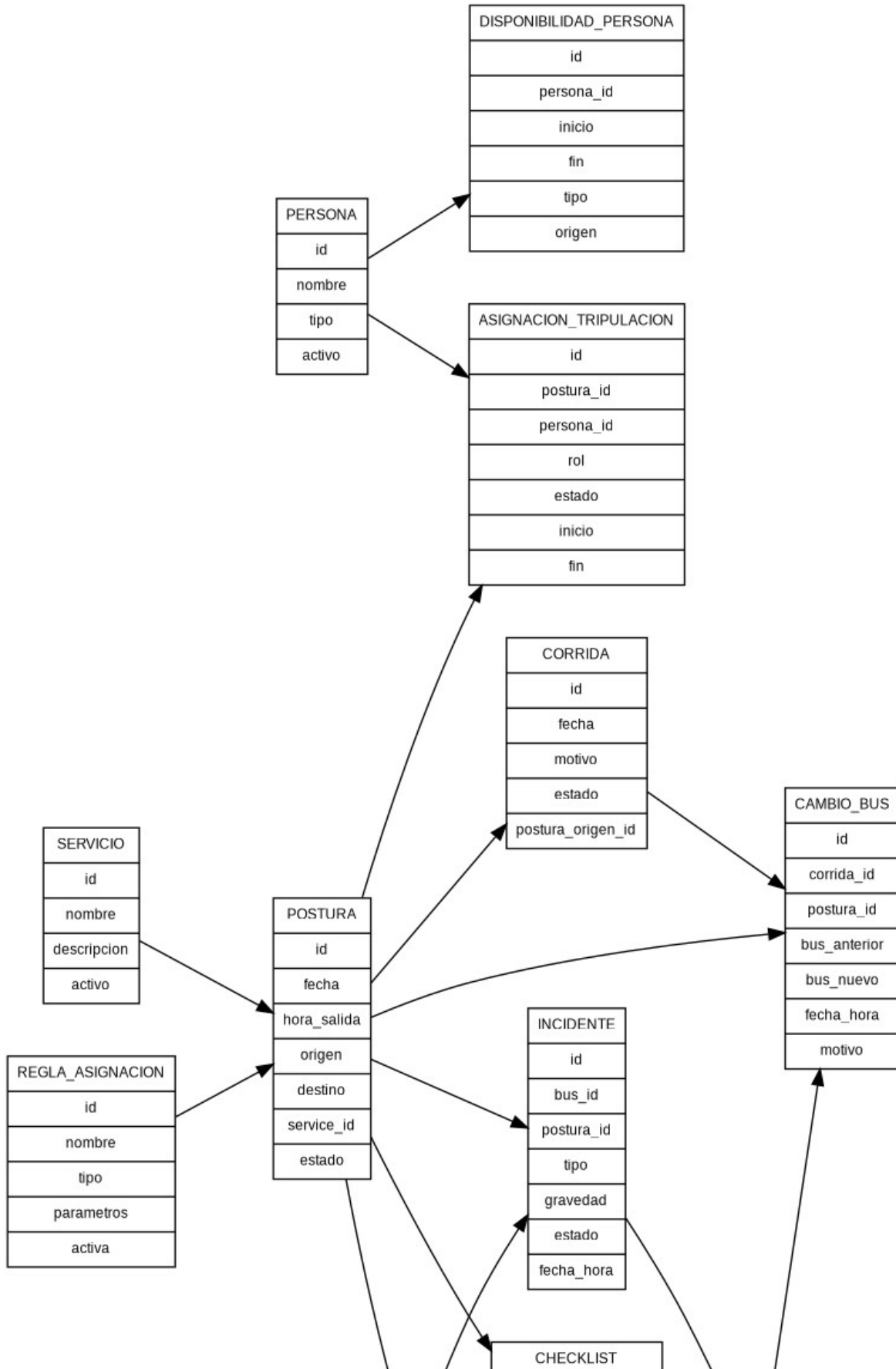
Capa	Tecnología	Motivo
Frontend	React/Next.js o Django templates + HTMX	Responsive; seleccionar según experiencia del equipo. Para un MVP rápido, Django + HTMX puede reducir complejidad.
Backend	Django + Django REST Framework	Modelo robusto, permisos, ORM, APIs e integración sencilla.
BD	PostgreSQL	Transacciones, integridad, consultas y crecimiento.
Async	Celery + Redis (cuando se requiera)	Notificaciones, tareas programadas, procesamiento no bloqueante.
Archivos	S3 compatible / almacenamiento cloud	Fotos de fallas, documentos y adjuntos.
Autenticación	Sesiones/JWT según frontend	Roles y control de acceso.
Despliegue	Docker + Linux + Nginx + HTTPS	Reproducible y portable.
Observabilidad	Logs estructurados + métricas + alertas	Diagnóstico de producción.
Integraciones	Adapters/clients REST, webhooks o proveedor oficial	Evitar acoplar el dominio a proveedores.

DECISIÓN DE ARQUITECTURA

Para este proyecto recomiendo comenzar como un monolito modular Django + PostgreSQL. No partir con microservicios: el dominio todavía está en descubrimiento y un monolito bien separado permite evolucionar sin complejidad operacional innecesaria.

9. Modelo de datos (ERD conceptual)

MODELO DE DATOS — NÚCLEO DEL MVP

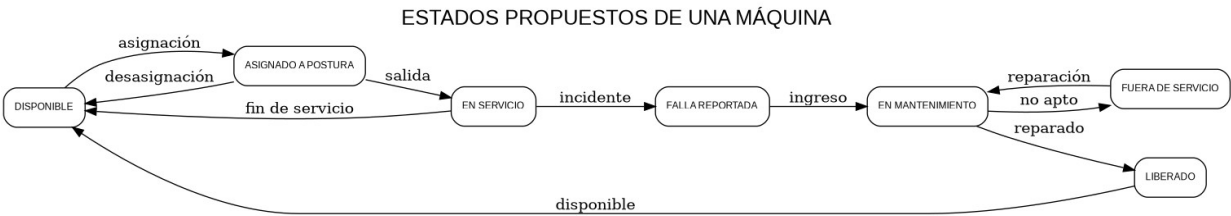


El siguiente modelo es conceptual; los nombres de campos son una base para el diseño físico.

9.1. Entidades principales

Entidad	Propósito
Bus	Máquina física y su estado operativo.
Servicio	Tipo de servicio comercial/operacional: Semi Cama, Salón Cama, Cama Premium, etc.
Postura	Instancia programada de un servicio para fecha/hora/origen/destino.
AsignacionBus	Relación histórica entre postura y máquina.
Persona	Base común para conductores/asistentes; puede separarse en tablas especializadas si el dominio lo requiere.
AsignacionTripulacion	Relación entre postura y persona, con rol y estado.
DisponibilidadPersona	Ventanas de disponibilidad/no disponibilidad.
Corrida	Evento que agrupa cambios por indisponibilidad y mantiene la secuencia.
CambioBus	Detalle de reemplazo de máquina dentro de una corrida.
Checklist	Registro realizado al llegar a Santiago.
Incidente	Falla/aviso durante la ruta, distinto del checklist.
OrdenMantenimiento	Trabajo mecánico asociado a bus/falla.
ReglaAsignacion	Configuración del motor de reglas.

10. Modelo de estados



Los estados exactos deben ajustarse en el levantamiento. La regla clave es que un estado represente una condición operacional real y no simplemente una pantalla.

11. API y contratos principales

La API debe organizarse por recursos y acciones de negocio. Ejemplo de endpoints iniciales:

Endpoint	Objetivo
GET /api/posturas?fecha=2026-09-01	Listado de posturas del día
POST /api/posturas	Crear postura
GET /api/posturas/{id}	Detalle
POST /api/posturas/{id}/asignar-bus	Asignar/cambiar máquina
POST /api/posturas/{id}/asignar-tripulacion	Asignar persona/rol
GET /api/posturas/{id}/conflictos	Obtener conflictos
POST /api/corridas	Crear corrida
GET /api/corridas/{id}	Detalle e historial

POST /api/incidentes	Registrar falla
POST /api/checklists	Registrar checklist en Santiago
GET /api/mantenimiento/ordenes	Órdenes de mantenimiento
POST /api/mantenimiento/ordenes/{id}/liberar	Liberar bus
GET /api/buses/{id}/estado	Estado actual e historial
GET /api/auditoria?entidad=postura&id=...	Historial de cambios

12. Ejemplo de modelo Django inicial

```
from django.db import models
```

```
class Service(models.Model):
    name = models.CharField(max_length=100)
    active = models.BooleanField(default=True)

class Bus(models.Model):
    class State(models.TextChoices):
        AVAILABLE = "AVAILABLE", "Disponible"
        ASSIGNED = "ASSIGNED", "Asignado"
        ROUTE = "ROUTE", "En servicio"
        MAINTENANCE = "MAINTENANCE", "En mantenimiento"
        OUT_OF_SERVICE = "OUT_OF_SERVICE", "Fuera de servicio"

    internal_number = models.CharField(max_length=30, unique=True)
    license_plate = models.CharField(max_length=15, unique=True)
    service_number = models.CharField(max_length=30, blank=True)
    state = models.CharField(max_length=30, choices=State.choices)

class Postura(models.Model):
    class Status(models.TextChoices):
        DRAFT = "DRAFT", "Borrador"
        READY = "READY", "Lista"
        WARNING = "WARNING", "Con advertencia"
        BLOCKED = "BLOCKED", "Bloqueada"
        IN_SERVICE = "IN_SERVICE", "En servicio"
        CLOSED = "CLOSED", "Cerrada"

    service = models.ForeignKey(Service, on_delete=models.PROTECT)
    date = models.DateField()
    departure_time = models.TimeField()
    origin = models.CharField(max_length=120)
    destination = models.CharField(max_length=120)
    status = models.CharField(max_length=30, choices=Status.choices)

class BusAssignment(models.Model):
    postura = models.ForeignKey(Postura, on_delete=models.CASCADE)
    bus = models.ForeignKey(Bus, on_delete=models.PROTECT)
    assigned_at = models.DateTimeField(auto_now_add=True)
    unassigned_at = models.DateTimeField(null=True, blank=True)
    reason = models.CharField(max_length=255, blank=True)

class CrewAssignment(models.Model):
    postura = models.ForeignKey(Postura, on_delete=models.CASCADE)
    person_id = models.PositiveBigIntegerField()
    role = models.CharField(max_length=30) # DRIVER / ASSISTANT
    status = models.CharField(max_length=30, default="ASSIGNED")
```

13. Motor de reglas de asignación

La aplicación debe separar datos de reglas. Una regla no debería estar escondida en una vista o formulario.

Regla	Tipo	Resultado
Bus en mantenimiento	Bloqueo	No candidato para asignación.
Bus ya asignado a postura incompatible	Bloqueo	No asignar.
Conductor con postura superpuesta	Bloqueo	No asignar.
Descanso insuficiente	Bloqueo o advertencia configurable	Mostrar razón.
Exceso de jornada/días	Bloqueo o advertencia configurable	Mostrar razón.
Titular ya ocupado	Advertencia/Bloqueo según política	Proponer relevo.
Postura sin bus	Advertencia	Mostrar candidatos disponibles.
Postura sin tripulación completa	Advertencia	Mostrar puestos faltantes.
Cambio de bus	Auditable	Exigir motivo y registrar historial.

El motor debe devolver una estructura similar a:

```
{
  "allowed": false,
  "severity": "BLOCK",
  "reasons": [
    {"code": "REST_INSUFFICIENT", "message": "Descanso insuficiente"}
  ],
  "alternatives": [
    {"person_id": 123, "status": "AVAILABLE"}
  ]
}
```

14. Especificación del prototipo

El prototipo debe ser entendible por una persona no informática. La pantalla debe responder rápidamente: ¿qué posturas tengo?, ¿qué falta?, ¿por qué no puedo asignar?, ¿qué bus está en problema?, ¿qué tengo que hacer ahora?

Pantalla	Qué debe resolver
P01 — Dashboard Operaciones	Indicadores: posturas totales, completas, con advertencia, bloqueadas; buses disponibles/en mantenimiento; tripulación pendiente; corridas activas.
P02 — Planificación diaria	Timeline de posturas con hora, servicio, origen/destino, bus, tripulación y estado. Filtros por hora/servicio/estado.
P03 — Detalle de postura	Información completa + asignar máquina + asignar tripulación + conflictos + historial.
P04 — Asignar conductor	Lista de candidatos con    , motivo del estado y alternativa/relevo.
P05 — Corrida	Vista visual de la secuencia: postura original, bus

	reemplazo y posturas afectadas.
P06 — Flota	Listado de máquinas con estado, próxima postura, mantenimiento y ubicación si existe.
P07 — Incidente en ruta	Registro rápido desde celular: bus, postura, tipo, gravedad, ¿puede continuar?, contacto mecánico, fotos, observaciones.
P08 — Checklist móvil	Checklist al llegar a Santiago; respuestas, observaciones, fotos y envío.
P09 — Mantenimiento	Bandeja de fallas/órdenes, prioridad, mecánico, especialidad, pozo, estado, liberación.
P10 — Auditoría	Quién cambió qué y cuándo; especialmente posturas, buses, tripulación y corridas.

15. Reglas de UX para Operaciones

- No mostrar 20 campos si no son necesarios en el momento.
- Usar lenguaje del negocio: postura, máquina, relevo, corrida, pozo, servicio.
- Toda alerta debe explicar el motivo, no solo mostrar un color.
- Una acción peligrosa debe pedir confirmación y motivo.
- El sistema debe sugerir candidatos; el usuario autorizado decide cuando la política permita excepciones.
- La pantalla de planificación debe funcionar bien en PC, pero los componentes críticos deben ser responsive.
- Para celular, evitar tablas gigantes: usar tarjetas, filtros y detalle por postura.

16. Seguridad, roles y auditoría

Rol	Permisos iniciales
Administrador	Configuración, usuarios, catálogos, reglas y auditoría.
Jefe de Operaciones	Posturas, asignaciones, corridas, excepciones y coordinación.
Programador	Crear/editar/programar posturas y asignaciones según permiso.
Operaciones	Asignar buses/tripulación, gestionar cambios y corridas.
Jefe de Mecánicos	Ver fallas, asignar trabajos, liberar buses, informar disponibilidad.
Mecánico	Ver trabajos asignados y actualizar ejecución.
Conductor	Ver sus posturas/tripulación; registrar incidente y checklist al llegar a Santiago.
Consulta	Solo lectura.

- Autenticación obligatoria.
- Permisos por rol y, si hace falta, por acción.
- Auditoría para cambios de postura, bus, tripulación, corrida, mantenimiento y reglas.
- No almacenar credenciales de WhatsApp/GPS en código; usar secretos de entorno/secret manager.
- Backups de PostgreSQL y política de recuperación definida antes de producción.

17. Plan de implementación

Fase	Objetivo	Resultado
------	----------	-----------

Fase 0	Validación	Cerrar AS-IS, términos, reglas y datos maestros. Responsable: negocio + desarrollo.
Fase 1	MVP Operaciones	Usuarios, buses, servicios, posturas, asignación de bus, tripulación y conflictos.
Fase 2	Corridas	Historial de cambios, sustituciones y visualización de secuencia.
Fase 3	Mantenimiento	Checklist digital en Santiago, incidentes, órdenes, mecánicos y liberación.
Fase 4	Comunicaciones	Plantillas, registro y luego integración WhatsApp.
Fase 5	Pasajeros / ventas	Integración con sistemas de terminal y andén.
Fase 6	GPS	Integración con proveedor actual y monitoreo.
Fase 7	Optimización	Motor de recomendaciones, reportes, indicadores y automatizaciones.

18. Criterios de aceptación del MVP

- Una postura puede crearse sin bus y sin tripulación completa.
- Una postura puede recibir y cambiar una máquina sin perder el historial.
- El sistema detecta al menos los conflictos de asignación configurados.
- El usuario ve por qué una persona o bus no puede ser asignado.
- Se puede registrar una corrida y visualizar las posturas afectadas.
- El conductor puede registrar un incidente durante ruta desde celular.
- El checklist se puede completar al llegar a Santiago desde celular.
- Mantenimiento recibe inmediatamente el checklist digital.
- El jefe de mecánicos puede cambiar el estado del bus y liberarlo.
- Los cambios críticos quedan auditados.
- La aplicación es usable en PC y celular.
- El núcleo funciona aunque WhatsApp/GPS/sistema de ventas todavía no estén integrados.

19. Preguntas pendientes de validación

Tema	Pregunta
Corrida / reincorporación	Cuando el bus reparado vuelve, ¿se reincorpora automáticamente a la secuencia, queda como reserva o se reasigna a una postura específica?
Número de servicio	Confirmar exactamente qué representa el número de servicio asociado al bus y cómo se relaciona con hoja de ruta/postura.
Reglas laborales	Definir oficialmente horas de descanso, límites de días/jornada, reglas 10x5/7x7 y excepciones.
Aprobaciones	Definir qué cambios requieren autorización del jefe de Operaciones o jefe de Mecánicos.

Estados de bus	Validar catálogo exacto: disponible, asignado, en ruta, pozo, mantenimiento, fuera de servicio, etc.
Incidente en ruta	Definir qué datos mínimos debe registrar el conductor y cuándo el mecánico debe escalar a Operaciones.
WhatsApp	Identificar proveedor/API y definir si se desea envío automático, respuestas o solo registro.
GPS	Identificar proveedor/API de SINACH/monitoreo interno y condiciones de acceso.
Ventas	Identificar software de cada terminal y posibilidad de API/exportación.
Checklist	Obtener el formulario actual completo para convertirlo en checklist digital campo por campo.
Pasajeros/andén	Definir qué información se necesita en cada terminal y quién autoriza su comunicación.

20. Prompt maestro para entregar a un desarrollador o IA

Actúa como arquitecto de software senior y desarrollador full-stack. Construye un sistema web responsive/PWA para gestión y planificación operacional de una empresa de buses.

Dominio confirmado:

- Existe una entidad SERVICIO (ej. Semi Cama, Salón Cama, Cama Premium) y una entidad POSTURA (servicio concreto programado por fecha/hora/origen/destino).
- Las posturas se crean diariamente; pueden existir sin bus y sin tripulación completa.
- Operaciones asigna máquinas según disponibilidad/llegadas.
- Normalmente una postura tiene 3 personas: 2 conductores + 1 asistente.
- Existe conductor titular/fijo asociado a una máquina y se completan puestos con relieves.
- La asignación de tripulación tiene restricciones de disponibilidad, solapamiento, descanso, jornada/días y reglas contractuales; estas reglas deben ser configurables, no hardcodeadas.
- Lunes y martes: servicios mineros tienen prioridad; luego se considera la cercanía horaria de las posturas.
- Durante ruta, si una falla impide continuar, el conductor contacta por WhatsApp al mecánico más cercano. Esto es un INCIDENTE, no un checklist.
- El checklist se realiza únicamente cuando el bus llega a Santiago. Actualmente un digitador lo recibe, revisa, digita e imprime para Mantenimiento.
- El sistema debe digitalizar ese checklist y entregarlo directamente a Mantenimiento.
- Mantenimiento asigna trabajos por especialidad, gravedad y disponibilidad. El jefe de mecánicos libera el bus después de revisar y, a veces, probarlo.
- Si un bus no puede cumplir una postura, Operaciones realiza una CORRIDA: otra máquina cubre la postura y las posteriores pueden desplazarse. Nunca sobrescribir el historial.
- Existe GPS en tiempo real, pero el proveedor/API debe identificarse.
- Los pasajeros/andén provienen de sistemas de ventas de terminal; la integración queda para una fase posterior.

Objetivo:

1. Diseñar primero el modelo de dominio y reglas.
2. Crear Django + PostgreSQL como monolito modular.
3. Exponer API REST para frontend/integraciones.
4. Crear interfaz clara para Operaciones y mobile-first para conductores.
5. Implementar auditoría.
6. Mantener WhatsApp/GPS/Ventas como adaptadores desacoplados.
7. Antes de codificar reglas laborales, señalar cuáles necesitan confirmación del negocio.

Entregables:

- Arquitectura.
- ERD.

- Modelos Django.
- Migraciones.
- Servicios de dominio para asignación, conflictos y corridas.
- API.
- Permisos/roles.
- Pantallas P01-P10 descritas en este documento.
- Pruebas unitarias para reglas.
- Datos de prueba.
- Docker para desarrollo.
- README con instalación y decisiones.
- No inventar reglas que no estén confirmadas; marcar TODO/VALIDAR.

21. Hoja de validación para Operaciones

Esta sección debe usarse en una videollamada desde celular. No se necesita lenguaje técnico.

- ☐ ¿La creación de posturas ocurre así?
- ☐ ¿Puede existir una postura sin máquina?
- ☐ ¿Puede existir una postura sin tripulación completa?
- ☐ ¿Los 3 integrantes son normalmente 2 conductores + 1 asistente?
- ☐ ¿La prioridad de lunes/martes funciona así?
- ☐ ¿La asignación de máquina funciona así?
- ☐ ¿La asignación de tripulación funciona así?
- ☐ ¿Estos son los principales conflictos de conductores?
- ☐ ¿Durante ruta la falla crítica se comunica al mecánico más cercano por WhatsApp?
- ☐ ¿El checklist se realiza SOLO al llegar a Santiago?
- ☐ ¿El flujo digitador → Mantenimiento está correctamente representado?
- ☐ ¿La corrida está representada correctamente?
- ☐ ¿Qué ocurre exactamente cuando vuelve el bus reparado?
- ☐ ¿Qué información falta en una postura?
- ☐ ¿Qué información falta en el checklist?
- ☐ ¿Qué decisiones solo puede tomar el jefe de Operaciones?
- ☐ ¿Qué decisiones solo puede tomar el jefe de Mecánicos?

22. Resultado esperado del proyecto

El sistema no debe ser entendido como un reemplazo de Excel o de WhatsApp. Debe ser el sistema de control operacional que mantiene una única versión de la verdad sobre posturas, máquinas, tripulación, fallas, mantenimiento y corridas. WhatsApp puede seguir existiendo como canal de comunicación, pero la información importante debe quedar registrada y trazable en la plataforma.

ORDEN DE TRABAJO

Próximo paso recomendado: presentar primero el BPMN AS-IS al responsable de Operaciones y corregirlo. Una vez validado, usar este documento para construir el prototipo TO-BE de las pantallas P01-P05. Después de validar esas pantallas, recién cerrar el modelo físico de base de datos y comenzar el desarrollo.